

# TraceWidth: A Scoped Audit-Ready State-Trace Protocol with Exact Certificates

Anonymous submission

## Abstract

Reliable reasoning evaluation increasingly depends on tasks whose provenance, scoring, and claim boundaries can be audited after models and datasets change. TraceWidth addresses this gap for a narrow state-trace setting by generating compact transition programs with exact-answer certificates, parser-normalized integer scoring, and row-level evidence links. We evaluate the protocol in a no-training 1,664-row packet: three small public arithmetic calibration slices, 126 generated TraceWidth tasks from three same-skill procedural families, one Qwen/Qwen2.5-7B-Instruct open-weight negative control, and a deterministic certificate-execution ceiling. The measured result is diagnostic rather than method-winning: on TraceWidth, direct prompting scores 117/378, fixed state-trace prompting scores 115/378, self-consistency scores 116/378, and code/tool execution reaches 378/378; the public slices add 49/64 direct, 48/64 fixed-answer-prompt, and 9/16 self-consistency successes. TraceWidth therefore contributes an audit-ready benchmark record with exact certificates, parser evidence, source-family breadth, and explicit claim downgrades, not a leaderboard, broad external-generalization claim, or prompt-improvement result.

## Introduction

Static benchmark scores become hard to interpret when task instances circulate widely or when scoring cannot separate answer extraction from reasoning failure. Work on benchmark contamination and dynamic evaluation argues that benchmark claims need renewable tasks, explicit provenance, and audit records rather than only aggregate scores (Chen et al. 2025; Chen, Pusarla, and Ray 2025). TraceWidth follows that direction in a narrow state-trace setting: a generator produces compact transition systems, a certificate computes the exact integer answer, and every paper-facing claim is tied to scored evidence.

Live and code-evaluation benchmarks show one response to the same problem: refresh or newly release tasks so that models cannot rely on memorized public answers, and use objective scoring when possible (White et al. 2024; Jain et al. 2024). Broader suites such as MMLU remain useful for coverage, but they do not by themselves solve provenance, renewal, or exact replay for a specific reasoning mechanism (Hendrycks et al. 2020). Generated-benchmark work similarly shows that a generator is scientifically useful only when the task recipe, answer certificate, and audit

trail are visible enough for later checking (Zhang et al. 2026; Majurski and Matuszek 2025).

TraceWidth asks a narrower question than broad benchmark replacement: can one generated procedural packet keep the evidence trail tight enough that a negative run remains scientifically useful? State-trace tasks are small enough to inspect, deterministic enough to certify, and structured enough to vary along interpretable axes. The current run is therefore useful even though the API scores are mixed: it separates healthy benchmark mechanics from evaluated-interface failures without claiming cross-benchmark coverage.

That distinction matters for contamination-aware evaluation. A generated benchmark can be renewed, but renewal is useful only if the paper records what changed and what stayed fixed. TraceWidth records task metadata so a later run can separate a new seed, generator family, template holdout, and surface renaming. The current run uses these records for auditability and descriptive slicing; it does not claim that future generated tasks are contamination-free.

State traces are a compact testbed for this agenda. Each TraceWidth item asks a system to follow a sequence of state updates and return one final integer. The format is narrower than open-ended reasoning, but it exposes failure modes that are difficult to audit in free-form rationales: alias tracking, delayed dependencies, template shifts, and parser cleanliness. It also creates a clean distinction between benchmark mechanics and model performance. If deterministic execution cannot recover the answer, the task or scorer is wrong; if model conditions miss a cleanly parsed answer, the error belongs to the evaluated condition. This emphasis on executable scoring follows code-evaluation work that uses objective tests to reduce grading ambiguity, but TraceWidth applies the idea to generated state traces rather than programming tasks (Chen et al. 2021; Jain et al. 2024).

The current combined run contains 1,664 scored rows over 158 unique task IDs. The evidence packet has 1,278 real-model rows and 386 deterministic code/tool rows, including 504 open-weight rows from Qwen/Qwen2.5-7B-Instruct. Direct prompting scores 166/442 after adding the public arithmetic slices, fixed-answer prompting scores 48/64 on the public slices, TraceWidth fixed state-trace prompting scores 115/378, self-consistency scores 125/394, and deterministic code/tool execution scores 386/386 over-

all. Parser diagnostics mark the TraceWidth-family rows as 1,511 clean parses plus one parsed-with-extra-text row, while the public rows use final-marker, last-integer, or self-consistency-majority parse statuses. The model/backend rows split sharply: ‘gpt-4o-mini’ has 39/450 successes across prompt rows, ‘gpt-5-mini’ has 410/450, and Qwen/Qwen2.5-7B-Instruct has 5/378. These numbers support a scoped generated-benchmark diagnostic claim with tool-ceiling calibration, backend diagnosis, small public arithmetic calibration slices, and same-skill procedural family breadth, not a model-ranking or broad cross-benchmark claim.

The research gap is not another broad leaderboard; it is a reproducible way to publish a generated reasoning packet whose evidence remains inspectable when the result is negative. TraceWidth contributes that record in three parts. First, it provides a no-training state-trace construction with exact integer certificates and generation metadata. Second, it reports a locked evaluation of direct prompting, fixed state-trace prompting, self-consistency, deterministic execution, and small public GSM8K, MATH-500, and BBH calibration slices under documented model/backend settings. Third, it makes claim calibration part of the benchmark artifact: prompt-improvement, broad generalization, cross-benchmark coverage, contamination resistance, and leaderboard comparisons remain outside the result unless future rows support them.

The paper’s negative result also constrains the research story. The fixed trace instruction and self-consistency condition are evaluated interface variants, not successful methods. TraceWidth treats the first full run as a measurement of the benchmark protocol and the evaluated interface rather than an invitation to repackage a prompt ablation as a win.

## Related Work

Contamination-limited evaluation has motivated dynamic benchmark construction, rolling evaluation sets, and source-backed audit trails (Chen et al. 2025; White et al. 2024; Majurski and Matuszek 2025). Code and executable-task benchmarks similarly emphasize objective scoring and freshness, because generated or newly released tasks reduce the chance that benchmark answers are memorized (Jain et al. 2024; Chen et al. 2021; Du et al. 2025). TraceWidth shares their emphasis on exact scoring and reproducible task construction, while deliberately focusing on one scoped state-trace packet plus small public arithmetic calibration slices rather than broad external benchmark coverage in the current evidence packet.

Benchmark suites such as MMLU provide broad coverage, but breadth alone does not solve provenance or contamination concerns (Hendrycks et al. 2020). Recent generated-benchmark proposals further motivate explicit generation recipes and audit records (Zhang et al. 2026; Chen, Pularla, and Ray 2025). TraceWidth contributes a focused diagnostic instance in this family: generated state traces with exact answers and an explicit paper-facing boundary between supported observations and blocked claims.

The first related line is contamination-aware evaluation. Surveys and dynamic-benchmark studies document why

static test sets can overstate generalization when examples or solution patterns leak into training corpora (Chen et al. 2025). LiveBench addresses this by maintaining a challenging and contamination-limited evaluation stream (White et al. 2024). Dynamic code-reasoning work similarly uses mutations or fresh tasks to study benchmark reliability under contamination pressure (Chen, Pularla, and Ray 2025). TraceWidth adopts the same evidence discipline but does not yet claim contamination resistance because the present evidence covers generated families within one scoped state-trace skill rather than independent external benchmarks.

The second line is executable or objectively scored reasoning evaluation. LiveCodeBench frames newly released coding problems as a route toward contamination-free code evaluation (Jain et al. 2024). Codex-style code evaluation demonstrates how executable scoring can avoid ambiguous natural-language grading for a subset of tasks (Chen et al. 2021). Repository-level and dependency-understanding benchmarks extend this idea to larger software contexts (Du et al. 2025). TraceWidth is intentionally smaller: it tests state traces, not software engineering, but it preserves the exact-scoring principle.

The third line is procedurally generated or source-grounded benchmark design. Generated spatial-temporal reasoning and synthetic-data grounding work illustrate why a generator must expose its knobs and why benchmark claims must be tied to source evidence rather than only to a headline score (Zhang et al. 2026; Majurski and Matuszek 2025). MMLU remains a useful broad anchor for multitask evaluation but does not by itself provide renewable provenance (Hendrycks et al. 2020). TraceWidth’s contribution is therefore not breadth; it is a measured diagnostic protocol with explicit claim calibration.

TraceWidth also treats evaluation as a software artifact rather than only a dataset. Executable scoring reduces ambiguity but introduces responsibilities: parser behavior must be stable, generated answers must be certified, and aggregation must preserve the distinction between model rows and deterministic sanity checks (Chen et al. 2021; Jain et al. 2024; Du et al. 2025). The present paper adopts that view by reporting generator version, task counts, condition counts, parser status, paired comparison policy, and explicit no-claim gaps.

Finally, the paper differs from leaderboard-first benchmark proposals. Broad suites and live benchmarks are useful when the evaluated model panel, task families, and statistical comparisons support ranking (Hendrycks et al. 2020; White et al. 2024). The current evidence does not. The related-work lesson is methodological: generated benchmarks need a way to publish negative or incomplete runs without converting them into unsupported wins.

TraceWidth also inherits a tradeoff from exact-scoring evaluations: executable scoring is auditable, but it discards partial progress (Chen et al. 2021; Jain et al. 2024). That tradeoff is appropriate here because the research object is the benchmark protocol and the final-state answer, not chain-of-thought quality. The paper reports parser status and exact accuracy, while qualitative failure modes remain descriptive.

## Method

TraceWidth tasks define a small state-transition trace and ask for an exact integer answer after a sequence of operations. The generator controls factors such as integration width, operation count, aliasing, callbacks, and template family. The scorer compares a parsed integer answer to the generated ground truth. The benchmark is no-training: the evaluated systems receive prompts or deterministic tool access at inference time only.

The benchmark is intentionally narrow, but its narrowness is a feature for diagnosis. Open-ended reasoning benchmarks often mix task comprehension, planning, language generation, and answer extraction. TraceWidth removes much of that surface: the target answer is one integer, the scorer is exact, and the deterministic certificate can be executed independently of a model. This makes the result less broad than a general reasoning benchmark but more precise about where the evaluated interface fails.

Table 1 summarizes the paper-facing controls used in the full run. Width and operation count affect the amount of state that must be integrated. Alias and callback controls stress whether a model can preserve state identity across indirect links. Template and generator-family controls record held-out surface and source-family slices. Isomorphic variants preserve underlying transition structure under surface changes, which makes them useful for later contamination and invariance analysis.

The TraceWidth parser is deliberately narrow. It accepts one integer answer after normalization and rejects responses with multiple competing numbers or no integer answer. This matters because a benchmark with exact ground truth can still fail if parser behavior is unstable. The saved parser diagnostics make the parser contract auditable: explanatory responses with one integer can be scored, while ambiguous multi-number responses are rejected. In the TraceWidth portion of the combined run, 1,133 of 1,134 real-model rows reached the clean-parser path, so nearly all reported wrong TraceWidth answers are condition errors rather than parser failures. Public arithmetic rows use final-answer-marker, last-integer, or self-consistency-majority parse statuses and are reported separately.

The deterministic code/tool condition has a different interpretation from the prompting conditions. It is a ceiling

| Control     | Diagnostic role                       |
|-------------|---------------------------------------|
| Width       | Varies state integration load         |
| Operations  | Varies trace length                   |
| Aliases     | Tests reference tracking              |
| Callbacks   | Tests delayed dependencies            |
| Templates   | Records held-out surface slices       |
| Isomorphism | Records structure-preserving variants |

Table 1: Six TraceWidth controls define the full-run stress axes; causal difficulty claims require matched slice analyses rather than aggregate counts alone.



Figure 1: TraceWidth audit flow from generated task through certificate, parser, scorer, and claim audit to blocked claims.

and integration test: if deterministic execution fails, the generator, certificate, or scorer is suspect. Since code/tool execution scored 378/378, the run supports the claim that the generated tasks are executable and that the scorer can recover the exact answer. It is not a model reasoning baseline.

The generation process records task structure before any model answer is requested. A task instance has a seed-derived transition trace, a final integer answer, and metadata describing the stress axis. This metadata is the bridge between benchmark construction and later analysis: it lets the paper ask whether errors cluster around aliasing, callbacks, width, template changes, or isomorphic variants. In this draft, these fields are used descriptively and as evidence-routing keys for the claim graph.

One evaluation episode has four paper-facing steps. First, the harness loads a generated task and its hidden certificate. Second, an API condition receives a text prompt and returns a short answer, or the deterministic condition executes the certificate directly. Third, the scorer normalizes the response and extracts exactly one integer when possible. Fourth, the extracted integer is compared with the certificate answer and stored with task metadata. This design keeps prompting, parsing, and exact answer correctness separate.

The fixed state-trace prompt is intentionally modest. It does not add tools, retrieval, training, or hidden state. It asks the model to maintain the state trace more explicitly before returning the final integer. That design makes the ablation interpretable: if the prompt helps, the same task identifiers should improve under the fixed instruction; if it does not, the failure cannot be hidden behind a different task sample. In the current full run, the fixed prompt does not help.

Self-consistency is similarly constrained. It samples multiple short answers and votes over parsed integers, but it does not inspect rationales or use an external verifier. The goal is not to build the strongest possible solver; it is to test whether a standard inference-time sampling tactic changes exact integer accuracy on the same state-trace tasks. Since self-consistency remains one success below direct prompting at the pooled level, the paper reports it as a non-improvement rather than a robustness gain.

The prompt and sampling conditions are deliberately not optimized after seeing the full-run outcome. Retuning them on the same TraceWidth task packet would compromise the diagnostic value of the run, because a later positive result could reflect adaptation to this particular generated set. A future method paper should define a new interface, lock it before evaluation, and rerun matched rows. This draft instead reports the current interface honestly and uses the negative result to motivate that future design.

## Experimental Setup

The combined run contains 1,664 scored rows over 158 unique task IDs: 24 external public GSM8K tasks, 4 MATH-500 integer-answer tasks, 4 BBH multistep-arithmetic tasks, and 126 generated TraceWidth tasks from one scoped state-trace packet with three same-skill procedural families: state-trace, alias-mesh, and callback-queue. It crosses two hosted API backends, ‘gpt-4o-mini’ and ‘gpt-5-mini’, plus the open-weight Qwen/Qwen2.5-7B-Instruct backend where available. The active condition labels are direct prompting, public fixed-answer prompting, fixed state-trace prompting, self-consistency, and deterministic code/tool execution. These names are evaluated model/backend identifiers recorded for the active combined run. The paper-facing metric is exact integer match accuracy after parser normalization.

Direct and fixed state-trace prompting use short chat-completion calls with a 32-token response budget; the hosted runs use temperature 0.0 and the Qwen open-weight rows use the recorded open-weight sampling setting. Self-consistency uses three sampled answers per task and votes over parsed integers; the run records the same paper-facing condition row count, while the model-call count is higher. Deterministic code/tool execution does not call a model; it executes the generated certificate and is reported only as a ceiling and scorer-health check.

The 158 task IDs are not a single uniform random sample. They combine three small public arithmetic slices with the original state-trace family and two TraceWidth family-expansion components that exercise alias-mesh and callback-queue variants. Table 2 reports the row inventory after crossing these components with available models and conditions. The table is not a difficulty ranking, because each component can differ in more than one way; it is a provenance check showing that the full run covers public calibration slices and the intended TraceWidth stress axes.

| Task component         | Rows | Diagnostic purpose                                                                                                                |
|------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------|
| GSM8K public slice     | 96   | External public math-word-problem calibration over 24 tasks and two hosted backends                                               |
| MATH-500 integer slice | 28   | External public competition-math calibration over 4 integer-answer tasks, two hosted backends, self-consistency, and tool ceiling |
| BBH arithmetic slice   | 28   | External public multistep-arithmetic calibration over 4 tasks, two hosted backends, self-consistency, and tool ceiling            |
| State-trace family     | 936  | Original procedural trace family with width, operation, alias, callback, template, and isomorphic controls                        |
| Alias-mesh family      | 288  | Same-skill procedural family emphasizing indirect state links across 24 tasks                                                     |
| Callback-queue family  | 288  | Same-skill procedural family emphasizing delayed updates across 24 tasks                                                          |

Table 2: Combined-run component inventory over 1,664 scored rows; the packet adds three small public arithmetic calibration slices while TraceWidth families provide same-skill procedural breadth, not broad external-benchmark generalization.

Table 3 gives the source-family and method matrix in paper-facing terms. It covers every available family/source by the conditions executed for that source. Backend-specific totals are reported in the surrounding prose, and the TraceWidth prompt comparisons remain matched at 126 rows per model-condition cell for paired tests. The Qwen/Qwen2.5-7B-Instruct rows are treated as a measured open-weight negative control, not as a hosted-versus-open leaderboard.

All reported accuracies use exact integer match. The draft does not report cost-normalized accuracy, latency, token efficiency, or wall-clock comparisons, because those fields are not part of the completed evidence chain. It also avoids cross-hardware comparisons. The no-training condition means no model weights are updated; the benchmark construction, scoring records, and claim-audit artifacts are the research object in this draft.

Matching is used only where the data support it. Direct prompting, fixed state-trace prompting, and self-consistency are compared over matched TraceWidth model-task units, producing 378 paired units for the pooled real-model comparisons. Public direct versus fixed-answer prompting has 64 matched model-task pairs. The deterministic ceiling is also paired by task, but its interpretation is different: a significant difference between direct prompting and code/tool execution only says that exact execution solves tasks that

prompted models usually miss. It is not evidence that a model baseline is worse than another model baseline.

The open-weight panel is now present but narrow. Qwen/Qwen2.5-7B-Instruct contributes 504 rows: 3/126 direct, 1/126 fixed state-trace prompt, 1/126 self-consistency, and 126/126 deterministic code/tool execution. These rows are a measured negative control for one open-weight backend under the recorded settings. They do not support a broad hosted-versus-open ranking or a general claim about open-weight models.

The reporting unit is a scored row, not a prompt transcript or a generated task alone. This choice keeps the denominator explicit. A task contributes one row per evaluated model-condition cell that exists for its source family, and condition-level counts are computed from those rows. The combined run therefore has 158 unique task IDs but 1,664 scored rows. Real-model rows and deterministic rows are separated because they answer different questions: model rows measure evaluated-interface behavior, while deterministic rows check generator and scorer consistency.

## Results

Table 3 gives the family/source-by-method result for the active packet. The fixed state-trace prompt did not improve over direct prompting on TraceWidth: 115/378 versus 117/378, with matched exact  $p=0.625000$  over 378 model-task pairs. Self-consistency also did not improve over direct prompting on all rows where it was run: 125/394 versus 126/394, with matched exact  $p=1.000000$ . Public fixed-answer prompting likewise does not improve over direct prompting in its 64 paired rows. The deterministic code/tool ceiling reached 378/378 on TraceWidth and 386/386 overall, confirming that the generated tasks, public integer-gold rows, and exact scorer are executable without representing model reasoning performance.

The parser result is part of the result, not only a setup detail. In the TraceWidth packet, 1,133 of 1,134 real-model rows and all 378 deterministic rows reached the parsed-clean path; one real-model row was parsed with extra text. Therefore the low TraceWidth prompt-condition accuracies in Table 3 are not caused by broad answer-extraction failure. They indicate that the evaluated model conditions usually returned the wrong final integer even when the parser could extract a single answer. This distinction is central to TraceWidth’s design: it lets future work report answer errors separately from parsing errors instead of mixing them into one failure rate.

Figure 2 visualizes the same condition-level evidence with Wilson intervals. The visual takeaway is not a prompt win; it is the gap between pooled model prompting and the deterministic execution ceiling, plus the backend-capability split hidden inside the pooled prompt bars. On the TraceWidth paired rows, direct prompting is two successes above fixed state-trace prompting and one success above self-consistency at the pooled count level; on the public fixed-answer rows, direct is one success above fixed-answer prompting.

The model-specific rows sharpen the interpretation. For ‘gpt-4o-mini’, direct prompting and self-consistency each

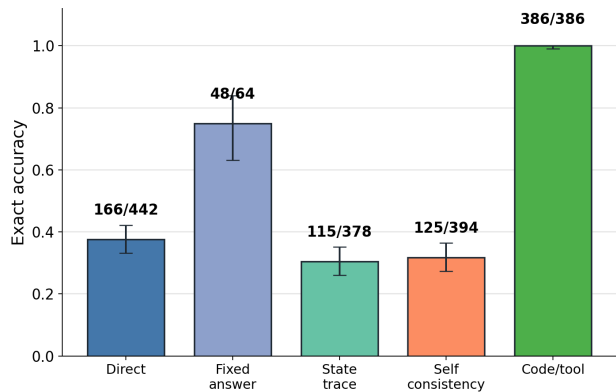


Figure 2: Data-derived combined-run condition accuracy: direct prompting scores 166/442 after adding public calibration slices, public fixed-answer prompting scores 48/64, TraceWidth fixed state-trace prompting scores 115/378, self-consistency scores 125/394, and deterministic code/tool execution reaches 386/386.

solve 1/126 tasks, while the fixed state-trace prompt solves 0/126. For ‘gpt-5-mini’, direct prompting solves 113/126 and the fixed state-trace and self-consistency variants each solve 114/126. For Qwen/Qwen2.5-7B-Instruct, direct solves 3/126, fixed state-trace prompting solves 1/126, and self-consistency solves 1/126. This split is evidence about evaluated backend capability under the same harness, not evidence for a new prompt method. It is sufficient, however, to rule out the pooled claim that the fixed state-trace instruction or self-consistency improved the evaluated interface in this run.

The parser-clean result is also a benchmark-health result. The parser probe accepts a clean integer answer and rejects ambiguous outputs; the TraceWidth rows then record 1,511 clean parses and one parsed-with-extra-text row. Together with the deterministic ceiling, this rules out task malformation and broad extraction failure for the TraceWidth headline counts. The 152 public arithmetic rows are scored by final-marker, last-integer, or self-consistency-majority parse statuses and are not used to make a TraceWidth parser-clean claim.

The family split is also diagnostic rather than a generalization claim. The original state-trace slice remains the largest family, while alias-mesh and callback-queue add 48 tasks and 576 scored rows. The added families do not reverse the negative prompt conclusion: alias-mesh is tied across all prompt variants, callback-queue differs by only one self-consistency success, and every family reaches a perfect deterministic certificate-execution ceiling. This pattern shows that the scoped packet exercises more than one procedural source family, while the supported conclusion remains narrower than broad external-benchmark robustness.

| Matched comparison          | Left    | Right   | Exact p  |
|-----------------------------|---------|---------|----------|
| Direct vs. fixed answer     | 49/64   | 48/64   | 1.000000 |
| Direct vs. fixed trace      | 117/378 | 115/378 | 0.625000 |
| Direct vs. self-consistency | 126/394 | 125/394 | 1.000000 |

Table 4: Paired full-run tests do not support a prompt-improvement claim for either fixed state-trace prompting or self-consistency.

## Analysis/Ablation

The main prompt ablation is negative. Fixed state-trace prompting did not help: it scored 115/378 against direct prompting’s 117/378, and the paired exact test shows no improvement. Self-consistency also did not become a stronger non-tool baseline; it scored 125/394 against direct prompting’s 126/394 on rows where self-consistency was run. These are not inconclusive positive trends. They are measured non-improvements under this interface and should be treated as diagnostic evidence about the current prompt and task format.

The parser result is cleaner than the model-result picture on TraceWidth. Parser diagnostics report 1,511 clean TraceWidth rows and one parsed-with-extra-text row. The weak ‘gpt-4o-mini’ slice, the remaining ‘gpt-5-mini’ misses, and the low Qwen prompt rows are therefore overwhelmingly reasoning or task-interface failures rather than parser failures. That distinction is important because a benchmark can otherwise hide extraction problems inside low accuracy.

The evidence boundaries are part of the result, not an afterthought. The full run supports parser stability, a deterministic execution ceiling, a backend split, and a measured Qwen negative control. It rejects a fixed-prompt improvement claim and a self-consistency improvement claim; it also does not support a hosted-versus-open leaderboard, broad benchmark generality, or contamination-resistance claim. The post-reference appendix records the full claim-boundary map.

| Claim group          | Status    | Paper-facing consequence                                                                                                                       |
|----------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Full combined packet | supported | 1,664 scored rows across six family files support a diagnostic benchmark record with three small public slices plus three TraceWidth families. |
| Prompt tests         | down      | Fixed trace prompting and self-consistency are reported as no-gain controls, not methods.                                                      |
| Backend split        | supported | ‘gpt-5-mini’ drives most prompt successes; ‘gpt-4o-mini’ and Qwen rows diagnose backend limits.                                                |
| Qwen taxon-omy       | supported | Qwen is a measured negative-control backend with mostly parse-clean wrong integers.                                                            |
| Scope bound-ary      | weak      | The run does not establish leaderboard status, broad external generality, or contamination resistance.                                         |

Table 5: Claim-graph summary for the 13 active claims: supported mechanics and backend diagnosis stay separate from downgraded prompt, leaderboard, generalization, and contamination claims.

The paired tests are intentionally exact and conservative. For direct versus fixed prompting, there are three baseline-only correct pairs and one candidate-only correct pair after matching by model and task. For direct versus self-consistency, the pooled counts are 117 direct successes and 116 self-consistency successes. These tests are not evidence that the prompts are equivalent in general; they are evidence that this run contains no measured improvement.

The failure pattern also informs benchmark design. Alias-count, callback-depth, width, operation-count, generator-family, template-family, and isomorphic-family slices are exported for descriptive diagnosis. The current paper does not treat them as causal difficulty effects; that would require matched comparisons that isolate one control from prompt length, operation count, answer range, and generator family.

The deterministic ceiling should not be overread. A 378/378 deterministic TraceWidth result and 386/386 overall deterministic result show that generated traces and public integer-gold rows can be executed and scored. They do not tell us whether a model can plan, trace, or compute the same answer unaided. They also do not validate every future generated task. The ceiling is a necessary sanity check for the harness; model claims must come from real-model rows, not from the deterministic certificate path.

The score-distribution diagnosis supports the same separation. Deterministic rows are constant at score 1.0 because they execute the certificate answers. Real-model rows vary in parsed answer and score, including one all-zero model-condition file for the fixed state-trace prompt on ‘gpt-4o-mini’ and one all-zero self-consistency file for Qwen/Qwen2.5-7B-Instruct. Those all-zero files contain varied parsed answers with no exact matches, so they are not fixtures or parser artifacts.

## Failure Cases

Among the 1,278 real-model rows, the run records 454 exact matches: 39/450 for ‘gpt-4o-mini’, 410/450 for ‘gpt-5-mini’, and 5/378 for Qwen/Qwen2.5-7B-Instruct. The remaining ‘gpt-5-mini’ TraceWidth prompt failures are parsed cleanly and include both original state-trace misses and new callback-queue misses. The Qwen taxonomy records 373/378 wrong prompt rows, 373 parse-clean wrong integers, six sign errors, and low-diversity answer modes. These are failure-localization axes, not proven causal difficulty factors; Appendix 7 and the failure-case appendix preserve the detailed map.

The other failure is interpretive. Isomorphic, generator-family, and template-heldout slices show that TraceWidth can label contamination-oriented comparisons, but this run does not prove contamination resistance. Deterministic code/tool execution is a scorer-health ceiling rather than a competing model baseline, the Qwen panel is one measured backend under one setting, and the prompt ablations are negative rather than hidden successes.

## Limitations and Scope

The evidence is intentionally bounded. The active packet includes three small public arithmetic calibration slices and three generated TraceWidth procedural families, but it is not a broad benchmark suite and does not establish generality across code, multimodal, long-horizon agent, or diverse mathematical reasoning tasks. The Qwen/Qwen2.5-7B-Instruct rows are a measured negative control for one open-weight backend under one recorded setting; they are not an open-weight model panel or a hosted-versus-open ranking.

The contamination evidence is also limited. TraceWidth records generator-family, template, isomorphic, and held-out source metadata, but the current run does not prove contamination resistance. A contamination claim would require sealed generation, larger held-out probes, and stability analysis across independently generated families. The exact-match metric is auditable but coarse: it does not measure partial reasoning quality, token efficiency, latency, or cost-normalized accuracy. These limits define the evidence required before the protocol can support broader method or leaderboard claims.

## Conclusion

TraceWidth currently supports a focused protocol/baseline claim: generated state-trace tasks, exact certificates, parser normalization, deterministic execution, and evidence-linked reporting can be combined into an auditable no-training reasoning benchmark record for one scoped procedural packet. In the completed combined run, the active evidence has 1,664 rows; the TraceWidth subset records 1,511 clean parses with one parsed-with-extra-text row, TraceWidth prompt conditions score 117/378, 115/378, and 116/378 at the pooled condition level, public calibration slices add 49/64 direct, 48/64 fixed-answer-prompt, and 9/16 self-consistency successes, and deterministic code/tool execution verifies a 386/386 executable ceiling. The strongest

evidence-backed claim is therefore about benchmark mechanics, backend-capability diagnosis, small public calibration slices, same-skill procedural family breadth, and claim calibration, not about a winning prompt method, cross-benchmark validation, or a broad benchmark suite.

The main scope boundaries are explicit. The current evidence covers three small external public arithmetic slices and three generated TraceWidth procedural families within one state-trace skill, so it improves source breadth but does not establish broad benchmark generality for code, multimodal, or long-horizon agent tasks. The open-weight rows are a measured negative control from one backend, Qwen/Qwen2.5-7B-Instruct, not a hosted-versus-open ranking or a general claim about open-weight models. The run records held-out and isomorphic slices, but it does not establish contamination resistance.

The prompt variants are intentionally simple and are not optimized solvers. A future method paper should lock a stronger interface before evaluation, run matched rows, and report whether the new interface improves over direct prompting without retuning on the same task packet. Finally, exact integer match is auditable but coarse, and the run does not report cost-normalized accuracy, latency-normalized accuracy, token efficiency, or wall-clock comparisons. These limits define the evidence needed before TraceWidth can support a leaderboard or broader systems comparison.

TraceWidth uses generated state-trace tasks and exact integer answers, not human-subject data, private user text, personal attributes, or sensitive decision targets. The main ethical risk is misinterpretation of a narrow generated benchmark as a broad model ranking. The paper addresses that risk by reporting the Qwen negative-control panel, the same-skill procedural family boundary, the negative prompt-ablation result, and the deterministic tool ceiling as a certificate check.

The immediate research value is procedural as much as numerical. TraceWidth shows how a generated benchmark can publish a completed negative run without losing audibility: the generator records task structure, the certificate supplies an exact answer, the parser exposes extraction status, and the claim graph prevents unsupported interpretations from entering the narrative. A stronger TraceWidth paper would add a broader open-weight panel, independent external benchmark families, and a locked solver interface evaluated on new matched tasks.

That audit trail also makes the negative result reusable. A later solver can keep the same task identifiers, rerun matched rows, and compare against the recorded direct, fixed-prompt, self-consistency, and tool-ceiling counts. A later benchmark-breadth study can add larger independent external benchmark families while preserving the distinction between real-model rows and deterministic certificate checks. The current prompt variants do not fail because the scorer is opaque: 1,133 of 1,134 TraceWidth real-model rows parse cleanly, and the deterministic certificate path solves every task. They fail because the evaluated text interfaces often return the wrong final integer. Future TraceWidth work should therefore be judged by observable row changes under locked interfaces, not by post hoc narrative repair of



the same evidence.

The same principle governs the paper’s claim graph. Each supported statement is tied to a saved table, summary, scored-row export, or generated artifact; each unsupported statement is either omitted or named as a boundary. This makes the paper less expansive than a leaderboard submission, but it makes the result inspectable. The contribution is the combination of renewable task generation, exact certificates, parser status, deterministic ceiling checks, model-row evidence, and explicit claim downgrades in one reproducible benchmark record. Under that record, a measured negative is still useful: it tells a future researcher where a new solver, model panel, or benchmark-family expansion must change the evidence rather than only the prose.

This is why the paper treats the prompt variants as evaluated conditions rather than as proposed methods. The fixed trace instruction and self-consistency policy are reasonable controls, but the saved rows show that neither changes the pooled outcome in the intended direction. Reporting that result protects the benchmark from a common failure mode: changing the story after the score is known. It also makes the next experiment concrete. A future system can improve the result only by solving more matched task rows, preserving parser cleanliness, and maintaining the certificate-execution ceiling across the same metadata fields. If those conditions hold, TraceWidth can support a stronger method claim; if they do not, the audit record will identify which part of the evaluation failed.

The current artifact is therefore best read as an audit-ready baseline for a narrow generated reasoning skill. Its value is not that the evaluated prompts are strong, but that the failure is measured under replayable conditions. The paper preserves task identifiers, model/backend labels, parser status, exact-match outcomes, paired comparison policy, and claim-boundary records so that later work can replace speculation with row-level deltas. That is the standard TraceWidth sets for itself: stronger claims require new locked rows, not stronger wording around old ones.

The completed run also defines a clean stopping point. The current evidence supports publication of the benchmark protocol, the scorer-health checks, the backend split, and the negative prompt-ablation finding. It does not support adding a stronger method label, a broad model ranking, or a contamination-resistance claim without new rows. Keeping that boundary visible is what makes the paper reviewable: the reader can see both the measured result and the exact evidence that would be needed to change it.

## Reproducibility and Artifact Map

The run uses no training. The relevant reproducibility facts are: generated state-trace tasks plus GSM8K, MATH-500, and BBH public slices, exact integer-match scoring, two hosted model labels plus Qwen/Qwen2.5-7B-Instruct, five active condition labels, 1,664 scored rows, 1,278 real-model rows, 386 deterministic code/tool rows, 504 open-weight rows, paired tests for prompt comparisons, and explicit claim downgrades for prompt improvement, broad generalization, leaderboard comparison, and contamination resistance. Public-facing regeneration should expose the bench-

mark generator, scorer, metric, seed policy, model/backend labels, condition labels, and scored-row schema.

The paper-facing reproducibility surface has three layers. The first layer is the task layer: generated state-trace items carry a generator version, seed-derived structure, a final integer answer, and stress-axis metadata. The second layer is the scoring layer: each evaluated row records the model/backend label, condition, parsed answer, parse status, exact-match score, and task identifier. The third layer is the analysis layer: aggregate condition tables, paired comparison tables, slice breakdowns, and claim-gap records are regenerated from the scored rows.

This section records no additional empirical claim beyond the body. Its purpose is to make the replay contract explicit: a reader should be able to regenerate condition summaries from scored rows, reproduce paired comparisons from matched model-task identifiers, and verify that unsupported claims remain absent when their evidence is missing.

| Evidence boundary                                                 | Paper-facing consequence                   |
|-------------------------------------------------------------------|--------------------------------------------|
| 1,511 clean TraceWidth parses plus one parsed-with-extra-text row | Parser stable on the TraceWidth packet     |
| Direct 117/378 vs. fixed 115/378                                  | No fixed-prompt improvement                |
| Direct 117/378 vs. self 116/378                                   | No self-consistency improvement            |
| Code/tool 378/378                                                 | Executable ceiling works                   |
| Qwen prompts 5/378                                                | Open-weight negative control               |
| Hosted/open comparison                                            | Not a leaderboard claim                    |
| Prompt improvement                                                | Requires a matched run showing improvement |
| Broad generality                                                  | Requires independent benchmark families    |
| Contamination resistance                                          | Requires larger sealed hold-out probes     |

Table 7: Nine evidence-boundary rows keep supported observations and blocked claims separate under the 1,664-row combined evidence.

## References

- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H. P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F. P.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A. N.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- Chen, S.; Chen, Y.; Li, Z.; Jiang, Y.; Wan, Z.; He, Y.; Ran,



D.; Gu, T.; Li, H.; Xie, T.; and Ray, B. 2025. Recent Advances in Large Language Model Benchmarks against Data Contamination: From Static to Dynamic Evaluation. arXiv:2502.17521.

Chen, S.; Pusarla, P.; and Ray, B. 2025. Dynamic Benchmarking of Reasoning Capabilities in Code Large Language Models Under Data Contamination. arXiv:2503.04149.

Du, J.; Liu, Y.; Guo, H.; Wang, J.; Huang, H.; Ni, Y.; and Li, Z. 2025. DependEval: Benchmarking LLMs for Repository Dependency Understanding. arXiv:2503.06689.

Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2020. Measuring Massive Multitask Language Understanding. arXiv:2009.03300.

Jain, N.; Han, K.; Gu, A.; Li, W.-D.; Yan, F.; Zhang, T.; Wang, S.; Solar-Lezama, A.; Sen, K.; and Stolica, I. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974.

Majurski, M.; and Matuszek, C. 2025. Grounding Synthetic Data Evaluations of Language Models in Unsupervised Document Corpora. arXiv:2505.08905.

White, C.; Dooley, S.; Roberts, M.; Pal, A.; Feuer, B.; Jain, S.; Schwartz-Ziv, R.; Jain, N.; Saifullah, K.; Dey, S.; Shubh-Agrawal; Sandha, S. S.; Naidu, S.; Hegde, C.; LeCun, Y.; Goldstein, T.; Neiswanger, W.; and Goldblum, M. 2024. LiveBench: A Challenging, Contamination-Limited LLM Benchmark. arXiv:2406.19314.

Zhang, Q.; Wang, Y.; Yang, R.; Wu, L.; Fan, J.; Yao, S.; Zhang, J.; Zhou, T.; Li, H.; Shi, R.; Li, Y.; and Qian, C. 2026. PRISM: A Benchmark for Programmatic Spatial-Temporal Reasoning. arXiv:2605.19382.

where errors concentrate in the recorded metadata, but not a causal claim that one control alone explains the miss.

The isomorphic, generator-family, and template-holdout slices should be read with the same caution. They demonstrate that the harness can label and score structure-preserving variants and held-out source or surface forms. A contamination claim would require sealed generation, sufficient rows per family, and stability analysis across held-out and in-family tasks.

## Reproducibility Checklist

The submission package includes the benchmark generator, scored-row exports, exact integer-match scorer, active combined-evidence manifest, source-family labels, model/backend labels, condition labels, paired-test table, and claim-evidence map. No training is used. The paper reports the evaluated hosted backends, the Qwen/Qwen2.5-7B-Instruct open-weight negative-control rows, deterministic certificate execution, row counts, task counts, parser statuses, and unsupported-claim boundaries. The package does not rely on private user data, human-subject data, sensitive attributes, or hidden human annotations. Public release should preserve the task schema, seed policy, scorer, result TSV schemas, and artifact digests so that every headline count in the paper can be regenerated from scored rows.

## Failure-Case Replay Details

The descriptive failure localization uses saved task metadata rather than inferred causes. The refreshed family-expansion packet includes both original state-trace misses and new callback-queue misses for ‘gpt-5-mini’. The Qwen negative-control taxonomy separately records 373 wrong prompt rows, all parse-clean, with answer-mode concentration and six sign errors. This supports a statement about

| Family/source   | Backend     | Tasks | Metric           | Budget dec.             |       | Direct | Fixed cond. | Self-cons.   | Tool ceiling | Diagnostic role                                                                                            |
|-----------------|-------------|-------|------------------|-------------------------|-------|--------|-------------|--------------|--------------|------------------------------------------------------------------------------------------------------------|
| GSM8K           | gpt-4o-mini | 24    | Exact final int. | 256 tok, $T = 0$        | 17/24 | 17/24  | –           | –            | –            | External public math slice; fixed condition is an answer-format prompt                                     |
| GSM8K           | gpt-5-mini  | 24    | Exact final int. | 256 tok, $T = 0$        | 23/24 | 22/24  | –           | –            | –            | External public math slice for source-breadth calibration                                                  |
| MATH-500        | gpt-4o-mini | 4     | Exact final int. | 256 tok, $T = 0$ ; SC=3 | 1/4   | 1/4    | 1/4         | <b>4/4</b>   | <b>4/4</b>   | Public integer-answer competition-math slice; tiny calibration only                                        |
| MATH-500        | gpt-5-mini  | 4     | Exact final int. | 256 tok, $T = 0$ ; SC=3 | 4/4   | 4/4    | 4/4         | <b>4/4</b>   | <b>4/4</b>   | Public integer-answer competition-math slice for parser/source breadth                                     |
| BBH arith-metic | gpt-4o-mini | 4     | Exact final int. | 256 tok, $T = 0$ ; SC=3 | 0/4   | 0/4    | 0/4         | <b>4/4</b>   | <b>4/4</b>   | Public multistep-arithmetic slice; tiny calibration only                                                   |
| BBH arith-metic | gpt-5-mini  | 4     | Exact final int. | 256 tok, $T = 0$ ; SC=3 | 4/4   | 4/4    | 4/4         | <b>4/4</b>   | <b>4/4</b>   | Public multistep-arithmetic slice for parser/source breadth                                                |
| State-trace     | gpt-4o-mini | 78    | Exact int.       | 32 tok, $T = 0$ ; SC=3  | 1/78  | 0/78   | 1/78        | <b>78/78</b> | <b>78/78</b> | Original generated trace family with width, operation, alias, callback, template, and isomorphism metadata |
| State-trace     | gpt-5-mini  | 78    | Exact int.       | 32 tok, $T = 0$ ; SC=3  | 71/78 | 71/78  | 71/78       | <b>78/78</b> | <b>78/78</b> | Same source family; hosted backend drives most real-model successes                                        |
| State-trace     | Qwen-2.5-7B | 78    | Exact int.       | 32 tok, $T = .7$ ; SC=3 | 2/78  | 1/78   | 0/78        | <b>78/78</b> | <b>78/78</b> | Open-weight negative-control slice on the original source family                                           |
| Alias-mesh      | gpt-4o-mini | 24    | Exact int.       | 32 tok, $T = 0$ ; SC=3  | 0/24  | 0/24   | 0/24        | <b>24/24</b> | <b>24/24</b> | Expansion stressing indirect state links                                                                   |
| Alias-mesh      | gpt-5-mini  | 24    | Exact int.       | 32 tok, $T = 0$ ; SC=3  | 24/24 | 24/24  | 24/24       | <b>24/24</b> | <b>24/24</b> | Same source family; all evaluated conditions solve this slice for this backend                             |
| Alias-mesh      | Qwen-2.5-7B | 24    | Exact int.       | 32 tok, $T = .7$ ; SC=3 | 0/24  | 0/24   | 0/24        | <b>24/24</b> | <b>24/24</b> | Open-weight negative-control slice on indirect-link tasks                                                  |
| Callback-queue  | gpt-4o-mini | 24    | Exact int.       | 32 tok, $T = 0$ ; SC=3  | 0/24  | 0/24   | 0/24        | <b>24/24</b> | <b>24/24</b> | Expansion stressing delayed updates                                                                        |
| Callback-queue  | gpt-5-mini  | 24    | Exact int.       | 32 tok, $T = 0$ ; SC=3  | 18/24 | 19/24  | 19/24       | <b>24/24</b> | <b>24/24</b> | Same source family; prompt variants differ by one task                                                     |
| Callback-queue  | Qwen-2.5-7B | 24    | Exact int.       | 32 tok, $T = .7$ ; SC=3 | 1/24  | 0/24   | 1/24        | <b>24/24</b> | <b>24/24</b> | Open-weight negative-control slice on delayed-update tasks                                                 |

Table 3: Family/source-by-backend method matrix over 1,664 scored rows with metric and decoding budget in the table: three small public arithmetic slices calibrate source breadth, and every TraceWidth source family reaches the deterministic certificate ceiling while real-model prompting remains a scoped diagnostic result rather than a broad cross-benchmark claim.

| Qwen diagnostic row        | Count   | Interpretation                                                      |
|----------------------------|---------|---------------------------------------------------------------------|
| Prompt exact matches       | 5/378   | Negative-control performance under the recorded open-weight setting |
| Parse-clean wrong integers | 373/378 | Failures are overwhelmingly not answer-extraction failures          |
| Sign-error slice           | 6/378   | Descriptive error tag, not a causal difficulty claim                |
| Direct prompt successes    | 3/126   | Best Qwen prompt condition remains low                              |
| Code/tool ceiling          | 126/126 | Task certificates execute cleanly for the same task set             |

Table 6: Qwen taxonomy paired with the stratified backend result: low Qwen prompt accuracy is a measured negative control while the deterministic certificate path remains perfect.